

Lab5 GPU 编程项目 AI 使用记录

1. 前期调研：了解 HIP 与实验范围

我： 本次实验使用 AMD GPU 和 HIP。HIP 的线程组织与 CUDA 有什么对应关系？

AI： HIP 同样使用 grid、block 和 thread 三级线程组织。全局线程编号通常写为：

```
int idx = blockIdx.x * blockDim.x + threadIdx.x;
```

AMD Radeon 8060S 默认使用 wave32，一个 wavefront 包含 32 个线程。block size 选择 32 的整数倍可以减少最后一个 wavefront 的空闲 lane。实验中还需要关注 global memory、LDS、atomic、同步和 occupancy。

我： 六个基础实验主要应该观察什么？

AI： Vector Add 观察 block size 和带宽；Matrix Transpose 观察连续读与跨步写；Histogram 观察 global atomic 与 LDS 私有化；Reduction 观察树形归约和 wavefront shuffle；Monte Carlo 观察采样规模与归约；K-Means 观察距离计算和簇中心更新。

2. 部分题目解答：线程、访存和归约

我： $N=1000$ 时，block size 分别为 64、128、256、512，需要多少 grid？

AI： 使用向上取整：

```
grid_size = (N + block_size - 1) / block_size
```

对应 grid size 为 16、8、4、2，都会启动 1,024 个线程，其中 24 个线程因边界判断不执行计算。

我： 为什么 Vector Add 更受带宽影响？

AI： 每个元素只执行一次加法，却需要读取 $A[i]$ 、 $B[i]$ 并写回 $C[i]$ 。算术强度较低，所以在并行度足够时，性能主要受显存读写效率影响。

我： Matrix Transpose 为什么会出现非合并写？

AI： 输入下标为 `input[idy * cols + idx]`，相邻线程读取连续地址；输出下标为 `output[idx * rows + idy]`，相邻线程写地址相隔 `rows` 个元素，因此写入呈跨步模式。可以使用 shared-memory tile 将全局读写都转换为较连续的访问。

我： Reduction 为什么使用 `__syncthreads()`？

AI： 每一轮树形归约都会读取上一轮写入 LDS 的中间值。`__syncthreads()` 保证同一个 block 内所有线程完成当前轮写入后再进入下一轮。wavefront 内使用 shuffle 交换寄存器值时，可以减少部分 LDS 访问和 block 级同步。

3. 部分代码编写：基础 kernel

我： Vector Add kernel 应该怎样补全？

AI： 可以按一线程处理一个元素的方式编写：

```
__global__ void vector_add(const float* A,
                           const float* B,
                           float* C,
                           int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        C[idx] = A[idx] + B[idx];
    }
}
```

我： 如果数组很大，如何让每个线程处理多个元素？

AI： 可以使用 grid-stride loop：

```
int tid = blockIdx.x * blockDim.x + threadIdx.x;
int stride = blockDim.x * gridDim.x;

for (int i = tid; i < N; i += stride) {
    C[i] = A[i] + B[i];
}
```

这种写法允许固定数量的线程覆盖任意长度数组，也能摊薄线程调度开销。

4. PCFG GPU 方案咨询

我： PCFG 口令生成中，一个 PT 由固定 prefix 和大量候选 value 组成，是否适合 GPU？

AI： 候选拼接彼此独立，适合数据并行。可以把 prefix 和 values 打包为连续缓冲区，每个 GPU 线程处理一个 value，并写入独立输出槽位：

```
output[i] = prefix + values[i]
```

小任务可能无法抵消内存分配、H2D、kernel launch 和 D2H 成本，因此应保留 CPU 路径，并通过候选数量阈值决定执行设备。

我： 多个 PT 应该怎样处理？

AI： 可以将多个 PT 合并为批次，记录各 PT 的 prefix、value 偏移和候选数。GPU 线程根据全局索引确定所属 PT。批处理可以减少 launch 次数；若使用多个 stream，还可以尝试让数据准备和设备执行重叠。

我： 如何验证 GPU 结果？

AI： 至少检查三项：候选总数一致、输出顺序一致、整体哈希一致。调度模式还应检查：

```
GPU 候选数 + CPU 回退候选数 = 候选总数
```

5. 云端运行问题咨询

我： 编译时提示 `No rule to make target clean`，可能是什么原因？

AI： 常见原因是脚本没有在 Makefile 所在目录执行。可以在脚本开头加入：

```
cd "$(dirname "$0")"
```

然后使用 `pwd`、`ls` 和 `find` 确认实际目录结构。

我： 程序提示 `Cannot open testcases/2.in`。

AI： 说明测试输入不存在。项目中保留了 `geninput.py`，可以在 Lab4 目录运行：

```
python3 geninput.py --quick
```

生成 `testcases/2.in` 后再执行 benchmark。

6. 实验数据解释咨询

我： Vector Add 哪个 block size 最快？

AI： 根据十次实测均值，128 threads/block 的 kernel 时间最低，为 `0.0210 ms`。32-512 threads/block 的误差范围存在较多重叠，因此不能只凭单次最小值判断绝对最优配置。

我： sub-wavefront 是否有明显影响？

AI： block size 8、16、32 的均值分别为 `0.0548 ms`、`0.0446 ms` 和 `0.0188 ms`。8 和 16 小于 wave32，存在明显的空闲 lane，平均时间高于 wave-aligned 配置。

我： Reduction 的 occupancy 都是 100%，为什么时间仍不同？

AI： occupancy 只描述可驻留线程比例。归约还受同步次数、空闲线程比例、global atomic 数量和每线程工作量影响。补充扫描中，Tree 方案从 block size 64 到 1024 的均值由 `0.02234 ms` 增至 `0.02912 ms`；Multi-Element 方案在 512 threads/block 时达到 `0.01030 ms`。这说明相同 occupancy 不代表相同性能。

我： GPU 路径一定比 CPU 快吗？

AI： 不一定。小规模任务中，数据打包、传输和 launch 成本可能超过 kernel 计算收益。应使用相同任务和相同计时边界比较，并同时报告 H2D、kernel、D2H 和端到端 wall time。

7. 使用总结

本项目中，AI 主要提供以下辅助：

- 查询 HIP、wavefront、LDS、occupancy 等概念；
- 解释六个基础实验的算法和性能现象；
- 解答部分 Exercise；
- 提供 Vector Add、grid-stride loop 和 PCFG 批处理等代码建议；
- 根据报错信息提供排查思路和终端命令；
- 协助解释 block size、传输成本和 CPU/GPU 分流结果。